

Introduction to React Hooks

1. React Components

There are two ways to write React Components.

- Function Components
- Class Components

Feature	Function Components	Class Components
State	No	Yes
Life Cycle Methods	No	Yes
Props	Yes	Yes

2. React Hooks

React Hooks are special functions that are a new addition to React

They allow us to use **state** and other **React features**(lifecycle methods, context, etc) in the Function Components

2.1 Importance of React Hooks

- Easy to reuse logic between the components
- Components become simple and easy to understand
- Less lines of code
- No need of switching between Class & Function Components
- Developers and Companies are gradually adapting to React Hooks

2.2 Hooks for Various Purposes

React provides multiple hooks which can be used for different purposes

Hook	Purpose
------	---------

useState()

Add State to Function Components

`useEffect()` Execute Logic after the Component
render

`useContext()` Access the Context value

3. State Hook

State Hook is a built-in Hook that allows us to add state to the Function Components

Syntax: `const [currentState, setterFun] = useState(initialValue)`

`useState` accepts the **initial value** as an argument and returns an array with two values: the **current state** and a **function** to update it

In the Class Components, the state is an Object. Where in the Function Components, the state can be of **any data type** i.e, boolean, number, string, object, etc.

3.1 Adding State

For adding the state to Function Components, import the

`useState` from `react` and call it inside the component

File: `src/components/GreetUser/index.js`

```
import { useState } from "react"

import {
  MainContainer,
  ...
} from "../styledComponents"

const GreetUser = () => {
  const [name, setName] = useState("")

  return (
    <MainContainer>
    ...
    </MainContainer>
  )
}
```

export default GreetUser

In the above code, we provided an empty string as the initial value to

useState

3.2 Updating State

We can update the state using **setter function** by passing value/callback function as an argument

```
const [currentState, setterFun] = useState(initialValue);
```

Using Value: Update the state using value if the next state is independent of the previous state

```
setterFun(nextValue);
```

Using Callback function: Update the state using callback function if the next state is computed based on the previous state

```
setterFun((prevState) => nextState);
```

3.2.1 Updating State using Value

File: src/components/GreetUser/index.js

```

import { useState } from "react";

import {
  MainContainer,
  NameInput,
  MsgContent,
  NameText,
} from "../styledComponents";

const GreetUser = () => {
  const [name, setName] = useState("");

  const onChangeName = (event) => setName(event.target.value);

  return (
    <MainContainer>
      <NameInput
        type="text"
        placeholder="Your name"
        value={name}
        onChange={onChangeName}
      />
      <MsgContent>
        Hello <NameText>{name}</NameText>
      </MsgContent>
    </MainContainer>
  );
};

```

export default GreetUser;

Class Component:

- When the state updates, the component re-renders only the render method will be called

Function Component:

- When the state updates, the entire Component will be re-rendered

- Even if the entire component is re-rendered, React will **preserve** the state variables between the re-renders

4. Declaring Multiple state variables

We can declare multiple state variables by calling **useState** multiple times

```
const [var1, setVar1] = useState(initialValue);  
const [var2, setVar2] = useState(initialValue);  
const [var3, setVar3] = useState(initialValue);
```

5. Initializing State

Syntax:

```
const [currentState, setterFun] = useState(initialValue);
```

The initial value provided to the **useState()** can be of any data type

5.1 Providing Object as an initial value

As a best practice, we can provide Object as an initial value when we want to store values that tend to change together

Example:

```
const [comment, setComment] = useState({ name: "", commentText: "" });
```

5.2 Providing Array as an initial value

We can provide the array as an initial value to **useState()**

```
const [commentsList, setCommentsList] = useState([]);
```

6. Updating State

We can update the state using **setter function** by passing value/callback function as an argument

```
const [currentState, setterFun] = useState(initialValue);
```

6.1 Updating State using Callback function

Update the state using callback function if the next state is computed based on the previous state

```
setterFun((prevState) => nextState);
```

7. Effect Hook

React provides a built-in hook called

`useEffect` that allows executing logic after the component render
In general this

`useEffect` hook can be simply called as **Effect hook**
Using Effect Hook we can perform actions like,

- Making API Call
- Using Scheduler methods like `setInterval( )`
- DOM Manipulations, etc.

Syntax: `useEffect(effect);`

The `useEffect` accepts **effect** as an argument, where the effect is a **callback function**

7.1 Updating Browser title using Effect Hook

Using the **title** property provided by the DOM object we can update the Browser title

Import the

useEffect from the react and call it inside the component

File: src/components/Scoreboard/index.js

```
import {useState, useEffect } from 'react'
...
const Scoreboard = () => {
  const [score, setScore] = useState(0)

  useEffect(() => {
    document.title = `Score: ${score}`
  })
  ...

  return (
    ...
    <ScoreText>{score}</ScoreText>
    ...
  )
}

export default Scoreboard
```

- React keeps track of the effect and **executes** it **after** the **render**
- All the state variables and props are accessible to the Effect
- If you try to pass arguments to effect and access them, you'll get undefined values

```
useEffect((score) => {
  document.title = `Score: ${score}`
})
```

7.2 Accessing State values

If you access the new state value immediately after updating the state in Event handlers, you may not get the updated value because **state updates are asynchronous**

```
const onIncrement = () => {  
  setScore(prevScore => prevScore + 1)  
  document.title = `Score: ${score}`  
}
```

Use Effect Hook, if you want to access new state values immediately after updating the state

```
useEffect(() => {  
  document.title = `Score: ${score}`  
})
```

8. Rules of Hooks

While writing/using the hooks you should follow the below rules,

- **Rule 1:** Hooks should be called only at the top level
- **Rule 2:** Hooks should be called only inside React Function Components and Custom Hooks

React provides the **eslint-plugin-react-hooks** package that throws errors if we don't follow these rules

9. Persisting data while using React Hooks

9.1 Storing values in Local Storage

In `useEffect` we have access to the updated state values, so we write the logic for storing values inside it.

File: src/components/BmiCalculator/index.js

```
const BmiCalculator = () => {
  const [height, setHeight] = useState(170)
  const [weight, setWeight] = useState(60)

  useEffect(() => {
    document.title = `Your BMI: ${getBmi(weight, height)}`
    localStorage.setItem("height", JSON.stringify(height))
    localStorage.setItem("weight", JSON.stringify(weight))
  })
  ...
  return (...)
}

export default BmiCalculator
```

9.2 Getting values from the Local Storage

Once after getting the values from the Local Storage, provide them as the initial values for the

useState hook

File: src/components/BmiCalculator/index.js

```
const BmiCalculator = () => {
  const storedHeight = JSON.parse(localStorage.getItem("height"))
  const storedWeight = JSON.parse(localStorage.getItem("weight"))

  const [height, setHeight] = useState(storedHeight !== null ? storedHeight : 170)
  const [weight, setWeight] = useState(storedWeight !== null ? storedWeight : 60)

  useEffect(() => {
    ...
    localStorage.setItem("height", JSON.stringify(height))
    localStorage.setItem("weight", JSON.stringify(weight))
  })
  ...
}
```

```
}
```

```
export default BmiCalculator
```

10. Tips for using the Effect Hook

10.1 Using Multiple useEffects to Separate Concerns

Just like `useState`, you can also use multiple `useEffects` to **separate unrelated logic** into different effects based on what it is doing

File: `src/components/BmiCalculator/index.js`

```
const BmiCalculator = () => {  
  ...  
  useEffect(() => {  
    document.title = `Your BMI: ${getBmi(weight, height)}`  
  })  
  
  useEffect(() => {  
    localStorage.setItem("weight", JSON.stringify(weight))  
  })  
  
  useEffect(() => {  
    localStorage.setItem("height", JSON.stringify(height))  
  })  
  ...  
}
```

```
export default BmiCalculator
```

10.2 Optimizing Performance by Skipping Effects

In some cases, executing the effect after every render creates a performance problem, It can be optimized by **Skipping** the **effect** when it is not required

The `useEffect` accepts another argument called **Dependency Array**, using which we can control the execution of effect

Syntax: `useEffect(effect, [var1, var2, ...])`

- The values we pass to the dependency array are called dependencies, which can be state variables or props
- Effect will be executed whenever the dependencies are changed

Without Dependencies

The Effect executes after every render

```
useEffect(effect);
```

Passing Empty Dependency Array

The Effect executes only once after the first render

```
useEffect(effect, []);
```

Passing Dependencies

The Effect executes when dependencies are changed

```
useEffect(effect, [var1, var2,...])
```

```
...
```

```
const BmiCalculator = () => {
```

```
  ...
```

```
  useEffect(() => {
```

```
    document.title = `Your BMI: ${getBmi(weight, height)}`
```

```
  }, [height, weight])
```

```
  useEffect(() => {
```

```
    localStorage.setItem("height", JSON.stringify(height))
```

```
  }, [height])
```

```

useEffect(() => {
  localStorage.setItem("weight", JSON.stringify(weight))
}, [weight])
...
}

```

export default BmiCalculator

11. Making API Call while using React Hooks

In Function Component, we can use

Effect Hook to make an API Call after the component render
API Call using the

fetch() method works asynchronously, so we need to write the API Call logic in an **async function**

11.1 Effects should be Synchronous

It's not a good practice to make an effect asynchronous because it may lead to unexpected results

```

useEffect(async() => {
  await fetch(url, options)
  ...
})

```

If you need an asynchronous function, define and call the async function inside the Effect

```

useEffect(() => {
  const funName = async() => {
    await fetch(url, options)
    ...
  }
  funName()
})

```

12. Displaying different views

While making an API call, we need to display different views like Loading View, Success View, and Failure View

For displaying these views, we need to maintain the following values in the state

- Status
- Data
- Error Message

We use a single state variable with an **object** as an initial value to store these values as they tend to change together

File: src/components/Leaderboard/index.js

```
import { useState , useEffect } from "react"
...
const apiStatusConstants = {
  initial: "INITIAL",
  inProgress: "IN_PROGRESS",
  success: "SUCCESS",
  failure: "FAILURE"
}

const Leaderboard = () => {
  const [apiResponse, setApiResponse] = useState({
    status: apiStatusConstants.initial,
    data: null,
    errorMsg: null
  })
  ...
  return (...)
}

export default Leaderboard
```

Based on the API Status, we display different views

File: src/components/Leaderboard/index.js

```
import { useState , useEffect } from "react"
...
const Leaderboard = () => {
  const [apiResponse, setApiResponse] = useState({
    status: apiStatusConstants.initial,
    data: null,
    errorMsg: null
  })

  useEffect(()=>{...})
  ...
  const renderLeaderboard = () => {
    const { status } = apiResponse
    switch (status) {
      case apiStatusConstants.inProgress:
        return renderLoadingView()
      case apiStatusConstants.success:
        return renderSuccessView()
      case apiStatusConstants.failure:
        return renderFailureView()
      default:
        return null;
    }
  }
  ...
  return(...)
}

export default Leaderboard
```

13 Updating Object in the State

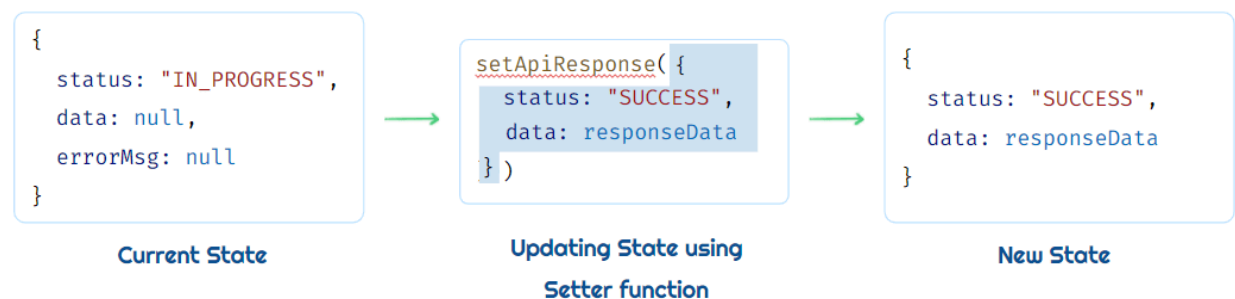
Class Components

- The Object provided to the `setState()` will **merge** into the current state object
- The newly created object contains all the keys and values



Function Components

- The Object provided to the setter function will **replace** the current state object
- The newly created object may not contain all the keys and values



While updating the state, use callback function and **spread operator** to get all the key values from the current state object

14. Using Scheduler methods

14.1 Displaying the current time

We use

Date Built-in Constructor function to display the current time


```
import { useState } from "react"

...

const Clock = () => {

  const [date, setDate] = useState(new Date())

  return (

    <ClockContainer>

      <Heading>Clock</Heading>

      <ClockImage src="IMAGE_URL" alt="clock" />

      <Time> {date.toLocaleTimeString()} </Time>

    </ClockContainer>

  )

}

export default Clock
```

14.2 Updating the time for every second

In general, it's a best practice to update the time after the component render

We schedule an interval inside the

useEffect for updating time after the component render

File: src/components/Clock/index.js

```
import { useState, useEffect } from "react"
```

```
...
```

```
const Clock = () => {
```

```
  const [date, setDate] = useState(new Date())
```

```
  useEffect(() => {
```

```
    const timerId = setInterval(() => {
```

```
      setDate(new Date())
```

```
    }, 1000)
```

```
  })
```

```
  return (<ClockContainer>...</ClockContainer>)
```

```
}
```

```
export default Clock
```

14.3 Avoiding unnecessary scheduling of intervals

In the code given in *section 1.2*, Effect executes multiple times, and many intervals are scheduled, but one Interval is enough to update the time for every second

The effect should execute and an interval should be scheduled only once, so pass **empty dependency array** as a second argument to the useEffect

File: src/components/Clock/index.js

```
const Clock = () => {

  const [date, setDate] = useState(new Date())

  useEffect(() => {

    const timerId = setInterval(() => {

      setDate(new Date())

    }, 1000)

    console.log("Effect executed and Interval scheduled")

  }, [])

  return (<ClockContainer>...</ClockContainer>)

}

export default Clock
```

15. Effect Hook

15.1 Cleanup function

The effect which we pass to the

`useEffect` can return a function called Cleanup function

Syntax:

```
useEffect(() => {
```

```
  // function body
```

```
  return () => {
```

```
    // cleanup
```

```
  }
```

```
}
```

In the Cleanup function we can perform actions like Clearing timers, Cancelling Network calls, etc

The Cleanup function will execute

- When the component unmounts
- Before the execution of next effect (*will be covered in next sessions*)

15.2 Handling Unmounting case

When a component is unmounted, the scheduled interval should be cleared to avoid unnecessary problems

To clear the scheduled interval when the component unmounts, call

`clearInterval()` in the Cleanup function

File: `src/components/Clock/index.js`

```
const Clock = () => {  
  
  const [date, setDate] = useState(new Date())  
  
  useEffect(() => {  
  
    const timerId = setInterval(() => {  
  
      setDate(new Date())  
  
    }, 1000)  
  
    ...  
  
    return () => {  
  
      clearInterval(timerId)  
  
      console.log("Cleanup Function executed")  
  
    }  
  
  }, [])  
  
  return <ClockContainer>...</ClockContainer>
```

```
}
```

```
export default Clock
```